

PRACTICAL-01

AIM:- CREATE TABLE USING SQL COMMANDS.

QUERY:-

```
CREATE TABLE Student (  
    Roll-no INT PRIMARY KEY,  
    Name VARCHAR(20),  
    Age INT,  
    Department VARCHAR(30),  
);
```

O/P:- TABLE will be created with 4 Attributes Roll-no, Name, Age and Department, where Roll-no will be primary key, which will uniquely identify each row of our table.

TABLE:

ROLL-NO	NAME	Age	Department
01	Aman	20	CSE

These data/record will be filled using INSERT INTO Command.

Syntax:

```
INSERT INTO Student (Roll-no, Name, Age, Department)  
VALUES (1, Aman, 20, CSE).
```

PRACTICAL - 02

AIM:- TO perform Insertion, updation and Deletion operations on a table.

SQL QUERIES:-

USING Practical - 1 table student

(Roll No, Name, Age, DOB)

(I) INSERTION

INSERT INTO students VALUES (1, 'Amo', 20, 'CSE');

INSERT INTO students VALUES (2, 'Riya', 21, 'ECE');

(II) UPDATION

UPDATE Students SET Age=12 WHERE Roll-NO=2;

(III) DELETION

DELETE FROM student ~~where~~ WHERE Roll-NO=1;

OUTPUT :

ROLL-NO	NAME	Age	Department
2	Riya	22	ECE

PRACTICAL-03

AIM:- TO USE SELECT Command for retrieving data from TABLE.

SQL QUERIES:- Using Student Table schema.
Student (Roll-No, Name, Age, department)

(i) SELECT * FROM Students;

This query will produce all column data, so it will provide whole table

O/P.

Roll-NO	Name	Age	Department
2	Riya	22	ECE

(ii) SELECT Name FROM students; // It will produce only one column, where column-name = Name.

O/P.

Name
Riya

- We can select whole table using (*).
- We can select any particular column by giving their name.
- We can select more than 1 column simultaneously.

PRACTICAL 04

AIM:- PERFORM PRIMARY KEY, CANDIDATE KEY, AND FOREIGN KEY USE IN THE TABLE-

SQL QUERIES:-

```
CREATE TABLE Department (  
    DeptID INT PRIMARY KEY,  
    DeptName VARCHAR(30) UNIQUE  
);
```

```
CREATE TABLE Employee (  
    EmpID INT PRIMARY KEY,  
    EmpName VARCHAR(50),  
    DeptID INT,  
    FOREIGN KEY (DeptID) REFERENCES Department (DeptID)  
);
```

O/P:

Department

DeptID	DeptName

Employee

EmpID	EmpName	DeptID	

Here DeptID is
Primary Key &
DeptName is Candidate
Key.

PRACTICAL - 05

AIM: Perform Joins (Outer Joins, Left Joins, Right Joins)

THEORY:- Joins are used in SQL to combine rows from 2 or more tables based on a related column between them.

Types

- i) LEFT JOIN :- Returns all records from the left table and the matched records from right table. If no match found, it will return null from the right side.
- ii) RIGHT JOIN :- Return all records from right table and the matched records from left table. If there is no match record found, null returned from ~~Right~~ left side.
- iii) OUTER JOIN :- Return all records when there is a match in either left or right table.

TABLES

Students

S.NO	Name	COURSE-ID
1	A	101
2	B	102
3	C	NULL

Courses

COURSE-ID	COURSE-NAME
101	DBMS
102	OS
103	CN

SQL QUERIES.

i) LEFT JOIN:-

```
SELECT Name, Course_name  
FROM Students
```

```
LEFT JOIN COURSES ON Students.COURSEID = COURSES.COURSE-ID;
```

o/p:

NAME	COURSE-NAME
A	DBMS
B	OS
C	NULL

PRACTICAL-06

AIM: NESTED QUERIES

THEORY: tables used - Employees

Emp-Id	name	salary
1	A	1000
2	B	2000
3	C	3000
4	D	4000

QUERY:

① Find employees earning more than Avg salary -

query: ②

SELECT name, salary

FROM Employees

WHERE salary > (SELECT AVG(salary) FROM employees);

op:

NAME	salary
C	3000
D	4000

query ②:- find the employees with highest salary

SELECT name, salary

FROM Employees

WHERE salary = (SELECT MAX(salary) FROM Employees);

o/p:

NAME	salary
D	4000

Left Right Join

query:

SELECT name, course-name
FROM students

RIGHT JOIN Courses ON student.course-id = Courses.course-id;

o/p:

NAME	Course-Name
A	DBMS
B	OS
⊙ NULL	CN

(iii) OUTER JOIN

QUERY:

SELECT name, course-name
FROM students

LEFT JOIN Courses ON students.course-id = Courses.course-id

UNION

SELECT Name, course-name

FROM students:

RIGHT JOIN Courses ON students.course-id = Courses.course-id;

o/p:

NAME	Course-Name
A	DBMS
B	OS
C	NULL
Null	CN

PRACTICAL - 01

AIM:- TO PERFORM UNION, INTERSECTION, EXCEPT OPERATION IN SQL

THEORY:-

TABLE A

ID	name
1	A
2	B
3	C

TABLE B

ID	name
2	B
3	C
4	D

① UNION:

Query:

```
SELECT * FROM A
UNION
SELECT * FROM B;
```

o/p:

ID	name
1	A
2	B
3	C
4	D

② INTERSECTION:-

Query: SELECT A.id, A.name
FROM A

INNER JOIN B ON A.id = B.id AND A.name = B.name;

o/p:

ID	NAME
2	B
3	C

EXCEPT

Query:

```
SELECT A.id, A.name  
FROM A  
LEFT JOIN B ON A.id = B.id AND A.name = B.name  
WHERE B.id IS NULL;
```

op:-

ID	name
1	A

PRACTICAL - 08

AIM:- To perform Aggregate function on a table using SQL.

THEORY:- tables used : employee

emp-id	name	salary
1	A	25000
2	B	30000
3	C	22000
4	D	28000
5	E	30000

QUERIES:-

1) ① TOTAL salary paid

SELECT SUM(salary) AS Total-salary FROM Employees;

O/P:

total salary
135000

2) Average salary

SELECT AVG(salary) AS Average-salary FROM Employees

O/P:

Average-salary
27000

3) MAXIMUM SALARY:-

SELECT MAX(salary) AS max-salary FROM Employees;

O/P:

Max-salary
30000

4) MINIMUM SALARY:-

SELECT MIN(salary) AS min-salary FROM Employees;

MIN-salary
22000

PRACTICAL-09

AIM:- TO CREATE A VIEW IN SQL

THEORY:- Table used:- Employees

emp-id	name	department	salary
1	A	HR	25000
2	B	IT	30000
3	C	HR	22000
4	D	IT	28000

QUERY: ① Create a view for IT department Employees

```
CREATE VIEW IT-employees AS  
SELECT name, salary  
FROM Employees  
WHERE department = 'IT';
```

② TO VIEW THE DATA

```
SELECT * FROM IT-employees;
```

O/p:

Name	salary
B	30000
D	28000

* PRACTICAL-10 *

AIM: TO demonstrate the use of grant and revoke commands in SQL.

THEORY. table used:- Employees

emp.id	name	salary
1	A	25000
2	B	30000

QUESTION:

① grant select permission to user 'student'

[GRANT SELECT ON employees TO 'student@localhost';]

This allows user 'student' to view data from employees table.

② GRANT, SELECT, INSERT PERMISSIONS:

[GRANT, SELECT, INSERT ON Employees TO 'student@localhost';]

NOW the user 'student' can view & insert data

③ [REVOKE INSERT ON Employees FROM 'student@localhost';]

This remove only the insert permission.